A Comparison between Classical and Quantum Machine Learning for Mobile App Traffic Classification

Vincenzo Spadari*, Idio Guarino†, Domenico Ciuonzo*, Antonio Pescapè*
*University of Napoli Federico II, †University of Verona
{vincenzo.spadari, domenico.ciuonzo, pescape}@unina.it, idio.guarino@univr.it

2024 IEEE/ACM Symposium on Edge Computing (SEC) | 979-8-3503-7828-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/SEC62691.2024.00053

***Abstract*—Network traffic analysis is essential for modern communication systems, focusing on tasks like traffic classification, prediction, and anomaly detection. While classical Machine Learning (ML) and Deep Learning (DL) methods have proven effective, their scalability and real-time performance can be limited by evolving traffic patterns and computational demands. Quantum Machine-Learning (QML) offers a promising alternative by utilizing quantum computing's parallelism. This paper examines QML's application in mobile traffic classification, comparing classical methods such as Multi-layer Perceptron (MLP) and Convolutional Neural Networks (CNNs) with Quantum Neural Networks (QNNs) using different embedding types. Our experiments, conducted on the MIRAGE-COVID-CCMA-2022 dataset, show that QNNs achieve competitive performance, indicating QML's potential for efficient large-scale traffic classification in future networks.**

***Index Terms*—Quantum Machine Learning, Deep Learning, Network Traffic Analysis, Network Traffic Classification.**

I. Introduction

Network traffic analysis plays a critical role in today's digital landscape [1, 2]. It enables proactive defense against cyber threats and improves network performance by identifying congestion points for timely intervention. This optimization enhances communication speed and tailors the network to specific traffic types such as audio, video, AR/VR, and gaming. Research has thus focused on developing sophisticated tools for traffic analysis, with key efforts in (i) *traffic prediction* [3, 4], i.e., understanding usage patterns to anticipate issues, (ii) *intrusion detection* [5], i.e., identifying anomalies by studying typical behavior, and (iii) traffic classification (TC) [6, 7], i.e., distinguishing different services and applications moving across the network.

In recent years, Artificial Intelligence, particularly *Machine-Learning* (ML), has become essential in TC, overcoming the *most of the limitations* of older methods, e.g., inference from encrypted packets. The dynamic nature of traffic, especially from *mobile apps*, reveals the limitations of traditional ML, making *Deep-Learning* (DL) the most effective approach to address these challenges [8].

DL techniques are thus increasingly used to monitor large data in real-time [9], but training and using them is *resource-intensive*, especially with evolving traffic patterns and threats. *Quantum Machine-Learning* (QML) offers a solution, leveraging quantum computing to improve data processing efficiency and speed. By blending classical data-driven techniques with quantum concepts, such as Quantum Neural Networks (QNNs), QML emerges as a solution for efficiently processing large volumes of data with greater speed and fewer resources [10, 11]. QML also parallels *TinyML* [12], which optimizes lightweight ML models on low-power devices. Combining QML and *TinyML* could also enable quantum-enhanced algorithms on minimal hardware, improving edge processing with lower energy use.

As a result, exploring QML in mobile app TC is promising due to its potential to handle data complexity, improve real-time analysis, enhance security, and offer long-term advantages over classical methods. While classical ML (including DL) methods remain effective, QML offers a *forward-looking approach* that could address the increasing demands and challenges of mobile networks more efficiently as quantum technologies mature.

vedi abstract

Accordingly, **our main contribution** is an initial inquiry into applying QML techniques to mobile TC, an area currently underexplored (to the best of our knowledge). We aim to provide an *in-depth comparison* between current DL methods and *newly-designed* QML-powered approaches. Using the public MIRAGE-COVID-CCMA-2022 dataset, we assess TC effectiveness at both the *app* and *user activity* levels, with a focus on the "earliness" of the TC outcome, namely labeling the app or activity as quickly as possible.

The paper is organized as follows: Sec. II reviews QML applications in network traffic analysis, focusing on TC. Sec. III explains our QML-based TC methodology, followed by the experimental setup in Sec. IV. Sec. V presents the results, and Sec. VI concludes with a summary and future directions.

II. Related work

Network Traffic analysis supports quality of service, infrastructure sizing, and security, with context and methodology be-

This work is partially supported by the "PNRR ICSC National Research Centre for High Performance Computing, Big Data and Quantum Computing (CN00000013)", under the NRRP MUR program funded by the NextGenerationEU. Also, this work is partially supported by the European Union under the Italian National Recovery and Resilience Plan (NRRP) of NextGenerationEU, partnership on "Telecommunications of the Future" (PE00000001 – program "RESTART").

2837-4827/24/$31.00 ©2024 IEEE
DOI 10.1109/SEC62691.2024.00053

ing key factors. Various methods, particularly ML, have been explored [2, 13, 14]. Meanwhile, quantum computing [15] has transitioned from theory to practice, offering solutions that enhance problem-solving speed [16, 17, 18]. This is particularly true for cryptography and complex optimization. This evolution has also fostered QML, now applied in diverse fields [19, 20, 21], including network traffic analysis.

Initial QML applications focus on traffic prediction [22], intrusion detection (anomaly detection [23, 24] and misuse detection [25, 26, 27, 28]), and TC [29, 30], which is central to our research and has been only recently explored.

Indeed, QML-based TC faces the challenge of handling a *multiclass classification* with a potentially large classification space, especially in the context of mobile apps. Specifically, through effective data preprocessing, Chao et al. [29] addressed both misuse detection and multiclass (attack) classification while addressing dataset imbalance. They compared their approach to classical Convolutional Neural Networks (CNNs) and a hybrid continuous variational quantum CNN, demonstrating that the quantum method not only overcame the limitations of unbalanced data but also outperformed classical models in both binary and multiclass scenarios. Abreu et al. [30] proposed QuantumNetSec, a framework integrating various QML techniques, including QNNs, quantum support vector machines, and quantum k-means. They implemented these algorithms on actual IBM hardware backends for the same binary and multiclass scenarios as [29], validating their findings against classical methods.

Positioning: while these studies demonstrate the potential of the field, *they all focus exclusively on network security scenarios.* Also, many rely on outdated datasets [26, 27, 28] with unrealistic traffic that does not reflect current traces. Additionally, they use *pre-processed (post-mortem) features* in their design [26, 27, 28, 29]. Last but not least, in most cases there is a limited evaluation setup, and only simplest scenarios (i.e., binary classification) are inspected with sufficient depth [25, 26, 27, 28]. Accordingly, to the best of our knowledge, *this is the first work to address the design of QML-based (early) traffic classifiers of mobile apps and corresponding user activities.*

III. Methodology

Background on Quantum Computing: Quantum computing is based on the principles of quantum mechanics, utilizing *quantum bits* (qubits) as the fundamental units of information. Quantum computers use the qubit to encode classical data into a quantum state [10, 24]. Unlike classical bits, which are either 0 or 1, a qubit can exist in both states, $|0\rangle$ and $|1\rangle$, simultaneously. This property, known as *superposition*, makes qubits highly expressive. In detail, the generic state of a qubit $|\psi\rangle$ can be formalized vectorially in a Hilbert space $\mathcal{H}$ and described by the following formula:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \tag{1}$$

where $\alpha, \beta \in \mathbb{C}$ and $|0\rangle, |1\rangle \in \mathcal{H}$, $\mathbb{C}$ being the set of complex numbers. In this representation, $|\alpha|^2$ (resp. $|\beta|^2$) is the probability of the qubit to stand in the basis state $|0\rangle$ (resp. $|1\rangle$). Another key property of the qubit is *entanglement*, whereby two interacting qubits remain bound even at long times and distances. This can be a way to express the correlation between features in a dataset encoded in a quantum state. A quantum circuit is made up of *quantum gates*, analogous to classical logic gates but operating on qubits. These gates, represented as unitary matrices in $\mathcal{H}$, typically produce spins.

QML-empowered classification: QML integrates quantum computing principles into traditional ML models. By utilizing quantum circuits to represent data and perform computations, QML algorithms can potentially explore complex patterns or solution spaces more efficiently. In contexts like TC, where large-scale data patterns must be identified, QML could outperform classical models by offering faster training times or enhanced accuracy through its ability to handle intricate, non-linear relationships that classical models might struggle with.

Quantum Neural Networks (QNNs) [31] represent one of several approaches within the field of QML, particularly suited for a classification task. The core of a QNN consists of *two* components: (*i*) *a feature map* and (*ii*) *an ansatz*. The feature map is a circuit that transitions data from classical encoding to a quantum state. This operation, typically called *quantum embedding*, can be achieved through several methods, e.g.:

- *Amplitude embedding* [20]: given a feature vector $\boldsymbol{x}$ of size N, it is represented in a quantum state $|x\rangle$ of $n = \lceil \log_2 N \rceil$ qubits in the following way:

$$|\boldsymbol{x}\rangle = \sum_{i=1}^{2^n} x_i |n_i\rangle \tag{2}$$

  where x_i is the i-th feature, whereas n_i is the i-th quantum state according to the computational basis[1]. Feature vector needs to be normalized ($\sum_i |x_i|^2 = 1$), while the case where the feature vector size is not a power of 2 can be accommodated by adopting padding strategies.
- *Angle embedding* [19, 20, 23]: for a feature vector $\boldsymbol{x}$ of size n, it returns a quantum state $|x\rangle$ of n qubits by applying qubit rotations as

$$|\boldsymbol{x}\rangle = \bigotimes_{i=1}^{n} (\sin(x_i)|0\rangle + \cos(x_i)|1\rangle) \tag{3}$$

  where $\bigotimes$ denotes the tensor product among the n qubits, highlighting that each feature is encoded into one qubit *without interactions*. Due to the one-to-one mapping between the size of the feature vector and the number of qubits (that could be prohibitive for large-dimensional input data), usually a compression strategy via a (trainable) dense layer is first used. In simple terms, a reduced feature vector $\boldsymbol{x}_r = \boldsymbol{h}_e(\boldsymbol{x}; \boldsymbol{\theta}_e)$ is fed into the embedding.

Conversely, the ansatz is a parametrized ($\boldsymbol{\theta}_c$) quantum circuit responsible for driving the inference task (in our case, a clas-

[1] E.g., if the encoding is made of 3 qubits, 1st vector is $|000\rangle$ state, 2nd is $|001\rangle$ state, etc.

sification problem) as well as the main object of optimization. The ansatz operation can be compactly described as

$$|y\rangle = \boldsymbol{U}(\boldsymbol{\theta}_c)|x\rangle \tag{4}$$

where $\boldsymbol{U}(\boldsymbol{\theta}_c)$ is a (composite) unitary transformation applied to the (n-qubit) input state $|x\rangle$. The ansatz can be unrolled as

$$\boldsymbol{U}(\boldsymbol{\theta}_c) = \boldsymbol{U}_1(\boldsymbol{\theta}_{c,1}) \circ \cdots \circ \boldsymbol{U}_L(\boldsymbol{\theta}_{c,L}) \tag{5}$$

in which L is the number of sequential layers composing the circuit, $\boldsymbol{\theta}_{c,\ell}$, $\ell = 1, \ldots, L$, denotes the set of trainable parameters of the ℓ-th quantum layer, and "$\circ$" is the composition operator. Each layer can be designed by exploiting quantum gates acting either (i) on single qubits (e.g., rotation gates) or (ii) on two or more qubits (e.g., CNOT gates).

Relevant information from the quantum circuit is extracted via a *measurement process* on the lines/wires, allowing the data to be returned in the classical form. A common choice is taking the expectation of the Pauli Z operator applied to generic qubit $|y_i\rangle = \alpha_i|0\rangle + \beta_i|1\rangle$, corresponding to $m_i = |\alpha_i|^2 - |\beta_i|^2$. The output, collected in the vector $\boldsymbol{m} \triangleq [m_1, \ldots m_n]$, can be further processed by another dense layer as $\boldsymbol{o} = \boldsymbol{h}_p(\boldsymbol{m}; \boldsymbol{\theta}_p)$ before feeding to a final softmax activation to obtain the confidence vector $\boldsymbol{p} \in [0,1]^C$ for the TC task. We stress that the input to the softmax (may be the quantum circuit measurement outcome or the output of a further dense layer) should be equal to the number of classes C.

Optimization of the (hybrid-model) parameters (i.e., $(\boldsymbol{\theta}_c, \boldsymbol{\theta}_p)$ or $(\boldsymbol{\theta}_e, \boldsymbol{\theta}_c, \boldsymbol{\theta}_p)$) is normally done via usual methods. Specifically, the QNN is trained to minimize a loss $\mathcal{L}(\boldsymbol{p}, \boldsymbol{p}_g)$ quantifying the discrepancy between the soft-output vector $\boldsymbol{p}$ and the one-hot encoded ground truth $\boldsymbol{p}_g$ over multiple training samples. Once trained, the obtained parameters define both the structure of pre-processing ($\boldsymbol{h}_e(\boldsymbol{x}; \boldsymbol{\theta}_e)$, if any) and post-processing steps ($\boldsymbol{h}_p(\boldsymbol{m}; \boldsymbol{\theta}_p)$), as well as the parameters defining the ansatz ($\boldsymbol{U}(\boldsymbol{\theta}_c)$).

IV. Experimental Setup

Dataset: The dataset used for the experiments is MIRAGE-COVID-CCMA-2022 [32]. It was collected during Apr.-Dec. 2021 using the MIRAGE capture tool [33] by students and researchers from the ARCLAB laboratory at the University of Napoli Federico II. The dataset includes mobile traffic data collected from Android devices, specifically related to *communication and collaboration apps* (CC apps).

The traffic data is stored in PCAP files and segmented into bidirectional flows (ref. to as biflows). A biflow groups all packets sharing the same 5-tuple $\langle IP_{src}, IP_{dst}, port_{src}, port_{dst}, L4\text{-}proto \rangle$, with src and dst interchangeable, and is labeled using log information. Each biflow is distinguished by both the *app* and the *activity* performed by the user. Precisely, the dataset includes 9 apps (i.e., `Discord`, `GotoMeeting`, `Meet`, `Messenger`, `Skype`, `Slack`, `Teams`, `Webex`, and `Zoom`) and 3 user activities, i.e., *Audiocall* (`ACall`), *Chat* (`Chat`), and *Videocall* (`VCall`). Herein, we focus on the *five* most popular apps—i.e., `Discord`, `Messenger`, `Skype`, `Teams`, and `Zoom`—and the *three* activities within the dataset (hence $C_{app} = 5$ and $C_{act} = 3$).

ML/QML-based Traffic Classifier workflow: In this work, we aim to assign each biflow to the app (viz. App-TC) or activity (viz. Act-TC) that generated it by analyzing its initial packets (aka *early* TC [34]). The workflow adopted is shown in Fig. 1. Specifically, we analyze how QML-based algorithms perform w.r.t. ML-based algorithms when tackling the aforementioned tasks.

Accordingly, for each biflow we employ the sequence of N_f transport-level header fields extracted from the first N_P packets. These fields include: (i) payload length (`PL`), (ii) packet direction (`DIR`), and (iii) TCP window-size (`TCPWIN`). For `DIR`, upstream and downstream directions are represented as $+1$ and -1 values, respectively, while `TCPWIN` is set to 0 for UDP packets.

To this end, we considered the following models:

1) *1-D Convolutional Neural Network* (1D-CNN): consisting of two convolutional layers (with 32 and 64 filters, respectively, kernel size of 3 and ReLU as activation), each followed by a max pooling operation of size 2. After convolutional layers, a flatten step is performed before a dense layer (128 units with ReLU activation) and a dropout (rate 0.5);
2) *Multi-Layer Perceptron* (MLP): featuring two dense layers (64 units with ReLU activation), each followed by a dropout (rate 0.5);
3) QNN *with Angle Embedding* (ANGE): hybrid classical-quantum neural network with an initial dense layer for feature extraction, followed by a 5-qubit circuit using angle embedding as the feature map and an ansatz made of strongly entangling layers ($L = 3$);
4) QNN *with Amplitude Embedding* (AMPE): a quantum neural network that directly receives a vector of 32 features, which is encoded using an amplitude embedding feature map onto 5 qubits, with the same ansatz as for ANGE.

All these classifiers are topped with a (softmax-activated) dense layer whose output size equals the number of classes C. To ensure the algorithms are properly fed with data we performed some preliminary operations. Each header field was normalized using a *per-feature MinMax* scaling process. Moreover, due to implementation purposes, we rearrange each TC object as an $N_f \cdot N_P$ array before feeding it to the ML/QML model (except for 1D-CNN).

Implementation details: the described algorithms were implemented using well-known Python libraries: Keras[2] and TensorFlow[3], for constructing ML/DL architectures, and PennyLane [4], for building and simulating QNNs. As for QNNs, the experimental simulations were conducted using the de-

[2]https://keras.io/
[3]https://www.tensorflow.org/
[4]https://pennylane.ai/

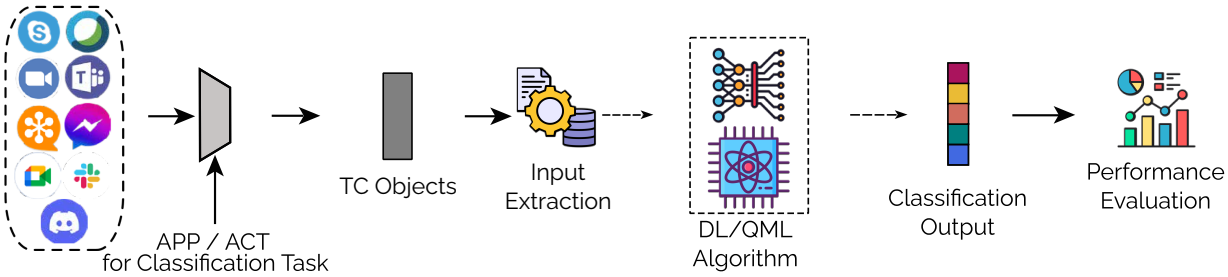


Fig. 1. Workflow of the methodology used for classifying mobile traffic via DL/QML approaches.

fault qubit-based simulator provided by PennyLane (i.e., `default.qubit`).

Evaluation setup: All traffic classifiers were evaluated by splitting the dataset with a $4:1$ ratio for training and testing. The training set was further divided with a $9:1$ ratio to create a validation set. The training was performed for 50 epochs—with a batch size of 32—for minimizing the categorical cross-entropy loss function and using the Adam optimizer—with a learning rate of 0.01, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-7}$. To avoid TC performance fluctuations due to randomness of training procedure, we trained (and tested) each model *five* times by fixing the seed.

V. Experimental Evaluation

Table I: Average performance, expressed in percentage, of the algorithms evaluated over 5 repetitions (App-TC). Results are presented as $mean \pm std.dev.$. For each metric, the best DL and QML approaches are highlighted in green and blue, respectively.

	Accuracy	Precision	Recall	F1-score
MLP	85.94 ± 1.17	88.79 ± 1.13	86.08 ± 1.37	86.47 ± 1.42
1D-CNN	91.01 ± 1.19	92.57 ± 1.17	91.37 ± 1.33	91.55 ± 1.36
AMPE	61.98 ± 1.45	63.94 ± 1.17	62.93 ± 1.24	62.95 ± 1.27
ANGE	85.00 ± 1.4	85.42 ± 1.58	85.29 ± 1.26	85.17 ± 1.4

Table II: Average performance, expressed in percentage, of the algorithms evaluated over 5 repetitions (Act-TC). Results are presented as $mean \pm std.dev.$. For each metric, the best DL and QML approaches are highlighted in green and blue, respectively.

	Accuracy	Precision	Recall	F1-score
MLP	53.57 ± 0.60	63.75 ± 11.66	36.26 ± 1.38	28.87 ± 2.46
1D-CNN	58.63 ± 0.57	64.79 ± 1.64	48.13 ± 1.54	45.73 ± 1.93
AMPE	53.93 ± 0.63	48.94 ± 4.46	41.0 ± 2.38	35.98 ± 2.98
ANGE	56.46 ± 0.70	54.81 ± 1.93	46.11 ± 1.80	44.38 ± 1.77

App Classification: we first compare the performance of DL and QML models in addressing App-TC. Accordingly, Tab. I reports the performance for each model in terms of Accuracy, and (macro) Precision, Recall, and F1-score. Specifically, 1D-CNN outperforms all other approaches, showing an improvement between $+3.8\%$ and $+5.3\%$ across all metrics compared to the second-best model (i.e., MLP). Conversely, AMPE performs the worst, with a gap of up to -29% compared to 1D-CNN.

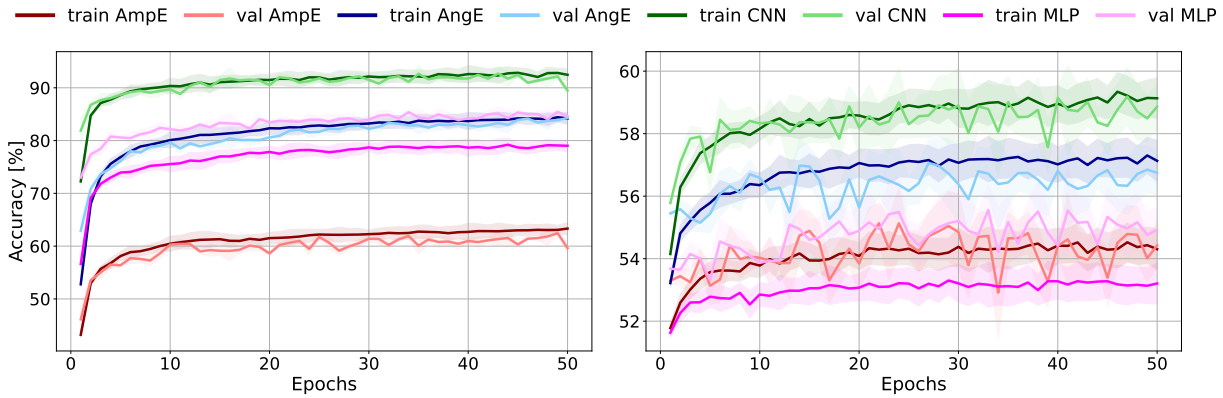
On the other hand, ANGE performs similarly to MLP, with an absolute gap of $\approx 1\%$ on all metrics (except for Precision). However, when examining the accuracy across epochs on both the training and validation sets (see Fig. 2a), we observe a considerable underfitting for MLP. In this case, the validation set consistently shows higher accuracy than the training set. In contrast, ANGE exhibits a similar trend on both the training and validation sets.

A significant trend indicates that `Skype` is frequently confused with `Teams` (see Fig. 3). This issue is evident even in the most effective method, 1D-CNN, which still manages to distinguish between other apps (Fig. 3a). It is also noticeable a mild tendency in classifying `Discord` as `Messenger`.

Activity Classification: similar to the previous analysis, we compare the performance of the DL and QML models in dealing with Act-TC. Tab. II reports the results for each model for all the aforementioned metrics. Notably, despite the reduced complexity of Act-TC (3 classes vs. 5 classes), its performance is significantly lower than that of App-TC, with a drop of -33% (resp. -46%) of Accuracy (resp. F1-score) between their best approaches. Furthermore, 1D-CNN and ANGE exhibit similar performance, with an absolute gap reduced to up $\approx 2\%$ for all metrics (except for Precision). Conversely, AMPE and MLP show worse performance, with MLP being the worst model overall (except for Precision). However, when examining the accuracy across epochs on both the training and validation sets (see Fig. 2b), we observe for a constant underfitting for MLP. In fact, we note a mean delta of $\approx 2\%$ between validation and training accuracy over epochs. On the other hand, ANGE exhibits minimal overfitting.

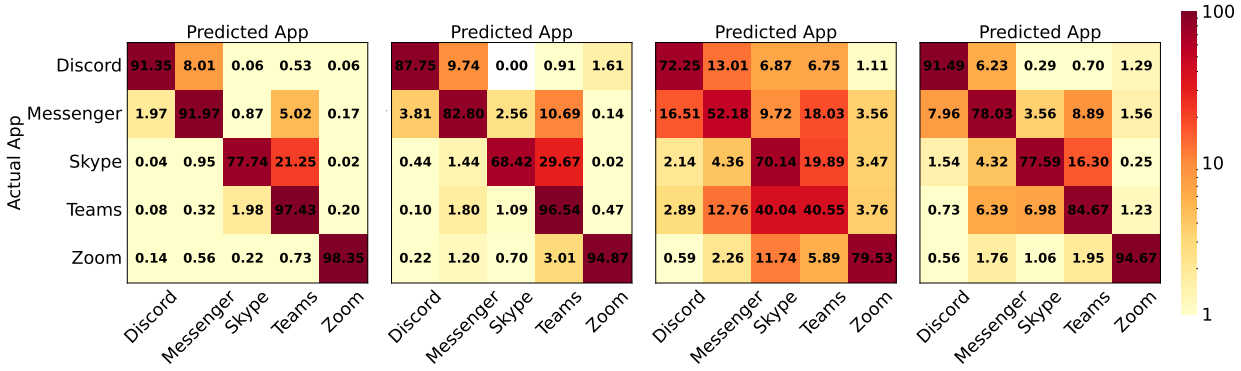
Across the algorithms examined, there is a general bias towards inferring `VCall` more frequently than other activities, as shown in Fig. 4. Specifically, `ACall` is often misclassified as `VCall`, whereas `VCall` is correctly identified most of the time. Conversely, `Chat` performs better than `ACall`, especially in the cases of 1D-CNN (Fig. 4a) and ANGE (Fig. 4d).

Soft-output analysis of Traffic Classifiers: finally, we focus on the soft output behavior of the best ML-based approach (1D-CNN) and the best QML-based approach (ANGE). The aim is to analyze the output provided by traffic classifiers



(a) App-TC (b) Act-TC

Fig. 2. Accuracy evolution across epochs on training (a) and validation (b) set for Act-TC task. Results refer to MLP, 1D-CNN, AMPE, and ANGE and are reported as $mean \pm std.dev$ across 5 repetitions.



(a) 1D-CNN (b) MLP (c) AMPE (d) ANGE

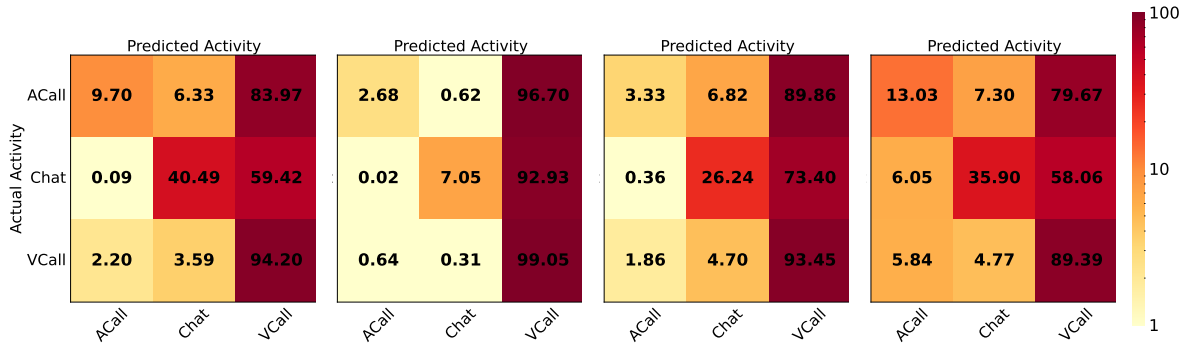
Fig. 3. Confusion matrices of (a) 1D-CNN, (b) MLP, (c) AMPE, and (d) ANGE w.r.t. App-TC task. Values are reported as $mean$ over 5 repetitions.

at a *finer-level* [8]. Figs. 5 and 6 show top-K accuracy (for $K \in 1, 2, 3$) and F1-score performance with a reject option (varying the threshold $\gamma \in [0, 1]$), respectively. In the first case, we define a correct classification if the actual class is in the top $K < C$ predictions (with $K = 1$ as standard accuracy). In the second, the classifier can abstain if the highest probability is below γ, allowing for improved performance with minimal reduction in classified instances (CR). Since apps typically generate multiple flows, distinctive flows can still aid identification without classifying every instance.

The results indicate that the performance gap between the 1D-CNN and ANGE narrows on the App-TC task when considering the top $K = 2$ most probable apps in the classification output (see Fig. 5). In contrast, the performance gap on Act-TC remains stable and minimal. This suggests that while the rich embedding of QNNs shows promise, further design enhancements are necessary to match the performance levels of DL methods like the 1D-CNN. Additionally, the reject option results (see Fig. 6) show that for the App-TC task, ANGE achieves nearly the same performance as the 1D-CNN, with an F1-score of approximately $\approx 95\%$, though it classifies 10% fewer biflows (90% vs. 80% CR for 1D-CNN vs. ANGE). In contrast, a different trend is evident in the Act-TC task. Both methods yield similar CRs at the same threshold γ, and both show an improvement in F1-score. However, 1D-CNN exhibits a sharper upward trend. In contrast, ANGE experiences a decline when $\gamma \in [0.75, 0.85]$, before rebounding to $\approx 80\%$ at $\gamma = 0.95$, compared to 90% for 1D-CNN at the same threshold.

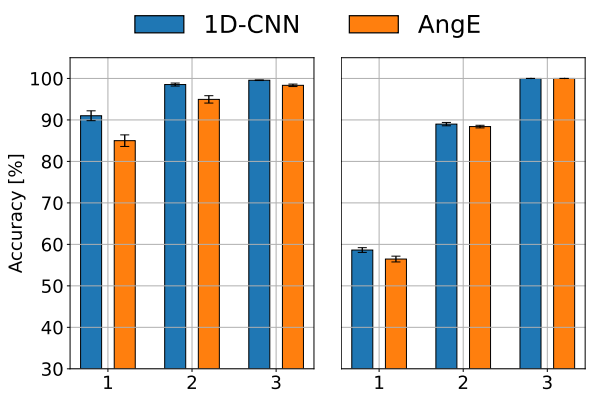
VI. CONCLUSION AND CRITICAL OUTLOOK

Exploring QML techniques for mobile TC, we draw several conclusions. First, 1D-CNN demonstrated the best performance in *App-TC* across all the metrics considered. Among quantum approaches, the ANGE model performed similarly to the MLP, indicating that QML techniques show promising potential in improving mobile apps recognition via network traffic analysis. Secondly, *Act-TC* did not exhibit a dominating



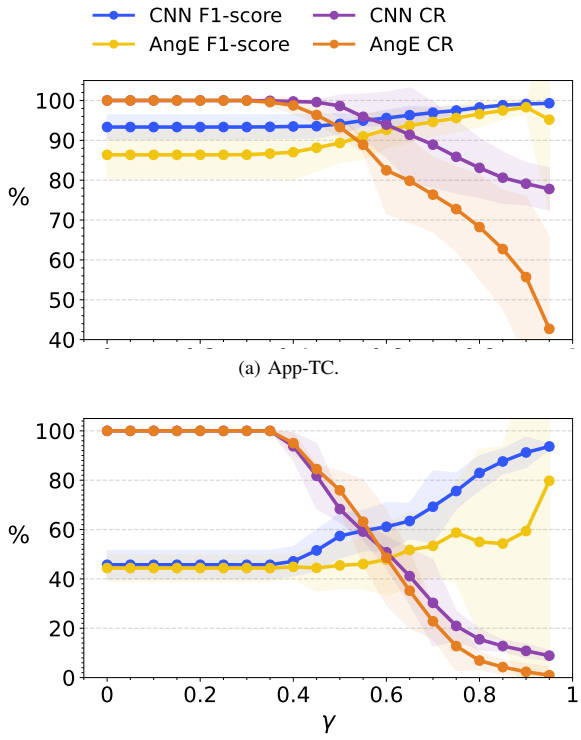
(a) 1D-CNN (b) MLP (c) AMPE (d) ANGE

Fig. 4. Confusion matrices of (a) 1D-CNN, (b) MLP, (c) AMPE, and (d) ANGE w.r.t. Act-TC task. Values are reported as $mean$ over 5 repetitions.



(a) App-TC (b) Act-TC

Fig. 5. Top-K Accuracy for 1D-CNN and ANGE related to (a) App-TC and (b) Act-TC as $K \in \{1, 2, 3\}$. Results are reported as $mean \pm std.dev$. across 5 repetitions.



(a) App-TC.

(b) Act-TC.

Fig. 6. F1-score and Classified Ratio set (CR) for 1D-CNN and ANGE on (a) App-TC and (b) Act-TC as the *reject option threshold* (γ) changes. Results are reported as $mean \pm std.dev$. across 5 repetitions.

algorithm, and although 1D-CNN showed even in this case a higher training and validation accuracy, ANGE stood the comparison in testing, proving its potential to solve these kinds of problems (including a satisfactory fine-grained behaviour). These findings contrast with prior studies in TC [29, 30] where QML consistently outperformed classical ML, but align with more recent (benchmark) research [35].

Future research could validate findings by deploying models on remote simulators or real quantum hardware, also testing their complexity (and quantum speedup). Applying these methods to diverse traffic datasets and tasks like malware classification may offer new insights. Using more advanced QML models and structured network inputs, such as multimodal data [32], could help uncover complex traffic patterns from mobile apps. Lastly, leveraging Explainable AI (XAI) [14] to analyze input-output relationships (and identify paths to QML improvement) is another promising direction.

REFERENCES

[1] D. Naboulsi, M. Fiore, S. Ribot, and R. Stanica, "Large-scale mobile traffic analysis: a survey," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 1, pp. 124–161, 2015.

[2] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Computing Surveys (CSUR)*, vol. 54, no. 6, pp. 1–35, 2021.

[3] G. O. Ferreira, C. Ravazzi, F. Dabbene, G. C. Calafiore, and M. Fiore,

"Forecasting network traffic: A survey and tutorial with open-source comparative evaluation," *IEEE Access*, vol. 11, pp. 6018–6044, 2023.
[4] I. Guarino, G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapè, "Explainable deep-learning approaches for packet-level traffic prediction of collaboration and communication mobile apps," *IEEE Open Journal of the Communications Society*, 2024.
[5] D. Chou and M. Jiang, "A survey on data-driven network intrusion detection," *ACM Computing Surveys (CSUR)*, vol. 54, no. 9, pp. 1–36, 2021.
[6] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1988–2014, 2018.
[7] J. Zhao, X. Jing, Z. Yan, and W. Pedrycz, "Network traffic classification for data fusion: A survey," *Information Fusion*, vol. 72, pp. 22–47, 2021.
[8] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapé, "Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges," *IEEE Transactions on Network and Service Management*, vol. 16, no. 2, pp. 445–458, 2019.
[9] N. Alqudah and Q. Yaseen, "Machine learning for traffic analysis: a review," *Procedia Computer Science*, vol. 170, pp. 911–916, 2020.
[10] M. Kalinin and V. Krundyshev, "Security intrusion detection using quantum machine learning techniques," *Journal of Computer Virology and Hacking Techniques*, vol. 19, no. 1, pp. 125–136, 2023.
[11] P. Rebentrost, M. Mohseni, and S. Lloyd, "Quantum support vector machine for big data classification," *Physical review letters*, vol. 113, no. 13, p. 130503, 2014.
[12] Y. Abadade, A. Temouden, H. Bamoumen, N. Benamar, Y. Chtouki, and A. S. Hafid, "A comprehensive survey on TinyML," *IEEE Access*, 2023.
[13] M. Abbasi, A. Shahraki, and A. Taherkordi, "Deep learning for network traffic monitoring and analysis (NTMA): A survey," *Computer Communications*, vol. 170, pp. 19–41, 2021.
[14] G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapé, "AI-powered Internet traffic classification: Past, present, and future," *IEEE Communications Magazine*, vol. 62, no. 9, pp. 168–175, 2024.
[15] R. Rietsche, C. Dremel, S. Bosch, L. Steinacker, M. Meckel, and J.-M. Leimeister, "Quantum computing," *Electronic Markets*, vol. 32, no. 4, pp. 2525–2536, 2022.
[16] T. Monz, D. Nigg, E. A. Martinez, M. F. Brandl, P. Schindler, R. Rines, S. X. Wang, I. L. Chuang, and R. Blatt, "Realization of a scalable Shor algorithm," *Science*, vol. 351, no. 6277, pp. 1068–1070, 2016.
[17] R. LaPierre and R. LaPierre, "Shor algorithm," *Introduction to Quantum Computing*, pp. 177–192, 2021.
[18] G.-L. Long, "Grover algorithm with zero theoretical failure rate," *Physical Review A*, vol. 64, no. 2, p. 022307, 2001.
[19] A. Senokosov, A. Sedykh, A. Sagingalieva, B. Kyriacou, and A. Melnikov, "Quantum machine learning for image classification," *Machine Learning: Science and Technology*, vol. 5, no. 1, p. 015040, 2024.
[20] T. Hur, L. Kim, and D. K. Park, "Quantum convolutional neural network for classical data classification," *Quantum Machine Intelligence*, vol. 4, no. 1, p. 3, 2022.
[21] K. Batra, K. M. Zorn, D. H. Foil, E. Minerali, V. O. Gawriljuk, T. R. Lane, and S. Ekins, "Quantum machine learning algorithms for drug discovery applications," *Journal of chemical information and modeling*, vol. 61, no. 6, pp. 2641–2647, 2021.
[22] W. Huang, J. Zhang, S. Liang, and H. Sun, "Backbone network traffic prediction based on modified EEMD and quantum neural network," *Wireless Personal Communications*, vol. 99, pp. 1569–1588, 2018.
[23] M. Hdaib, S. Rajasegarar, and L. Pan, "Quantum deep learning-based anomaly detection for enhanced network security," *Quantum Machine Intelligence*, vol. 6, no. 1, p. 26, 2024.
[24] A. Kukliansky, M. Orescanin, C. Bollmann, and T. Huffmire, "Network anomaly detection using quantum neural networks on noisy quantum computers," *IEEE Transactions on Quantum Engineering*, 2024.
[25] M. Hdaib, S. Rajasegarar, and L. Pan, "Quantum autoencoder frameworks for network anomaly detection," in *International Conference on Neural Information Processing*. Springer, 2023, pp. 69–82.
[26] C. Gong, W. Guan, H. Zhu, A. Gani, and H. Qi, "Network intrusion detection based on variational quantum convolution neural network," *The Journal of Supercomputing*, pp. 1–28, 2024.
[27] Z. Abou El Houda, H. Moudoud, B. Brik, and M. Adil, "A privacy-preserving framework for efficient network intrusion detection in consumer network using quantum federated learning," *IEEE Transactions on Consumer Electronics*, 2024.
[28] A. Gouveia and M. Correia, "Towards quantum-enhanced machine learning for network intrusion detection," in *IEEE 19th international symposium on Network Computing and Applications (NCA)*, 2020, pp. 1–8.
[29] S. Chao, G. Yang, and M. Nie, "Hybrid continuous variational quantum neural networks for network intrusion detection," in *Industrial Engineering and Applications*. IOS Press, 2023, pp. 366–378.
[30] D. Abreu, D. Moura, C. Rothenberg, and A. Abelém, "Quantumnetsec: Quantum machine learning for network security," *Authorea Preprints*, 2024.
[31] W. Li, Z.-d. Lu, and D.-L. Deng, "Quantum neural network classifiers: A tutorial," *SciPost Physics Lecture Notes*, p. 061, 2022.
[32] I. Guarino, G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapè, "Contextual counters and multimodal deep learning for activity-level traffic classification of mobile communication apps during covid-19 pandemic," *Computer Networks*, vol. 219, p. 109452, 2022.
[33] G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapé, "MIRAGE: mobile-app traffic capture and ground-truth creation," in *IEEE 4th International Conference on Computing, Communications and Security (ICCCS)*, 2019, pp. 1–8.
[34] L. Bernaille, R. Teixeira, and K. Salamatian, "Early application identification," in *ACM CoNEXT conference*, 2006, pp. 1–12.
[35] J. Bowles, S. Ahmed, and M. Schuld, "Better than classical? the subtle art of benchmarking quantum machine learning models," *arXiv preprint arXiv:2403.07059*, 2024.